

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN

ĐỒ ÁN II

XÂY DỰNG HỆ THỐNG
TỰ ĐỘNG SINH NGÂN HÀNG CÂU HỎI

SÁI HOÀI NAM

Nam.SH227137@sis.hust.edu.vn

Mã sinh viên 20227137

Chuyên ngành Toán - Tin

Giảng viên hướng dẫn: TS. Trần Ngọc Thăng

Chữ ký của GVHD

HÀ NỘI – 01/2026

NHẬN XÉT CỦA GIẢNG VIÊN

1. Mục tiêu

.....
.....
.....

2. Nội dung

.....
.....
.....

3. Đánh giá kết quả đạt được

.....
.....
.....

Hà Nội, ngày ... tháng ... năm 2025

Giảng viên hướng dẫn

TS. Trần Ngọc Thăng

Lời cảm ơn

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc đến thầy TS. Trần Ngọc Thăng, người thầy đã tận tình hướng dẫn, định hướng chuyên môn và luôn đồng hành cùng em trong suốt quá trình thực hiện đồ án. Sự nghiêm túc và tinh thần học thuật của thầy là nguồn động lực to lớn giúp em không ngừng cố gắng, hoàn thiện nội dung của đề tài "Xây dựng hệ thống sinh ngân hàng câu hỏi tự động" một cách hiệu quả.

Tuy đã nỗ lực hoàn thành tốt nhất trong khả năng của mình, nhưng không tránh khỏi những thiếu sót. Em rất mong nhận được những góp ý quý báu từ thầy để có thể hoàn thiện hoàn chỉnh đồ án về "Xây dựng hệ thống sinh ngân hàng câu hỏi tự động". Từ đó em có thể rút ra nhiều kinh nghiệm hơn cho các nghiên cứu tiếp theo của mình ạ

Em xin chân thành cảm ơn!

Hà Nội , ngày ... tháng ... năm 2025

Tác giả đồ án

Sái Hoài Nam

Tóm tắt nội dung báo cáo

Trong kỷ nguyên chuyển đổi số mạnh mẽ của nền giáo dục hiện đại, nhu cầu về nguồn học liệu số hóa, đặc biệt là các ngân hàng câu hỏi trắc nghiệm chất lượng cao phục vụ cho hệ thống thi cử trực tuyến, đang trở nên cấp thiết hơn bao giờ hết. Tuy nhiên, các phương pháp soạn thảo thủ công truyền thống hiện nay không còn đáp ứng được yêu cầu về tốc độ và quy mô, trong khi việc ứng dụng các mô hình trí tuệ nhân tạo tạo sinh thuần túy lại thường xuyên gặp phải vấn đề "ảo giác", dẫn đến những sai lệch nghiêm trọng trong các bài toán đòi hỏi tính chính xác tuyệt đối. Xuất phát từ thực tiễn đó, Đề án II với đề tài "**Xây dựng hệ thống sinh ngân hàng câu hỏi tự động**" đã tập trung nghiên cứu và triển khai một giải pháp công nghệ lai ghép tiên tiến theo kiến trúc **Neuro-Symbolic AI**, nhằm giải quyết triệt để bài toán khan hiếm dữ liệu luyện thi.

Phương pháp tiếp cận cốt lõi của đề án là sự kết hợp hài hòa giữa sức mạnh tính toán chính xác, tuân thủ chặt chẽ các quy tắc logic toán học của máy tính với khả năng xử lý ngôn ngữ tự nhiên linh hoạt của các mô hình ngôn ngữ lớn. Cụ thể, thành phần Ký hiệu (*Symbolic Component*) sử dụng thư viện **SymPy** để đảm nhận các tác vụ giải toán đại số và kiểm tra logic, đảm bảo kết quả đầu ra luôn chính xác tuyệt đối. Song song với đó, thành phần Mạng nơ-ron (*Neural Component*) sử dụng

Google Gemini API để thực hiện các tác vụ làm giàu ngữ nghĩa, viết lại lời dẫn đề bài cho tự nhiên và sinh ra các phương án nhiều dựa trên tư duy của người học.

Về mặt kỹ thuật, hệ thống được xây dựng hoàn chỉnh trên nền tảng web với kiến trúc Client-Server ba tầng. Phía máy chủ được phát triển bằng ngôn ngữ Python và Framework Flask, tích hợp cơ chế thực thi mã nguồn an toàn (*Sandbox*) cho phép tham số hóa linh hoạt các dạng bài toán từ đơn giản đến phức tạp. Cơ sở dữ liệu SQLite được thiết kế tối ưu theo mô hình lai, kết hợp giữa bảng quan hệ và định dạng phi cấu trúc JSON để quản lý hiệu quả các mẫu câu hỏi động. Phía người dùng cung cấp một giao diện trực quan, hiện đại, hỗ trợ soạn thảo và hiển thị công thức toán học chuyên sâu thông qua định dạng LaTeX, giúp giảm thiểu rào cản công nghệ cho giáo viên.

Kết quả đạt được của đề án là hệ thống phần mềm **MathBank** hoàn chỉnh với đầy đủ các chức năng nghiệp vụ từ quản lý mẫu câu hỏi, sinh tự động hàng loạt biến thể câu hỏi đến việc trộn và xuất bản đề thi theo chuẩn. Hệ thống đã chứng minh được tính hiệu quả trong việc tạo ra số lượng lớn câu hỏi trắc nghiệm thuộc các lĩnh vực Đại số và Giải tích với chất lượng đồng đều, đa dạng về ngữ cảnh và loại bỏ hoàn toàn sai số tính toán. Thành công của đề án không chỉ cung cấp một công cụ hỗ trợ đắc lực cho công tác giảng dạy mà còn mở ra hướng nghiên cứu tiềm năng về việc ứng dụng AI lai ghép trong lĩnh vực công nghệ giáo dục.

Hà Nội, ngày ... tháng ... năm 2025

Tác giả đề án

Sái Hoài Nam

Danh mục hình ảnh

3.1	Kiến trúc tổng thể	15
3.2	Sơ đồ kiến trúc dữ liệu	18
3.3	Dataflow quy trình sinh câu hỏi	21
3.4	Cơ chế Sandbox trong SolverService.execute()	22
3.5	Một số API của Template	24
3.6	Giao diện Dashboard	24
3.7	Giao diện Quản lý môn học	26
3.8	Quản lý Template	27
3.9	Giao diện chỉnh sửa và tạo mới Template	28
3.10	Giao diện danh sách câu hỏi	31
3.11	Giao diện Sinh câu hỏi	33
3.12	Giao diện Sinh đề thi	35
3.13	Xuất đề thi ra Latex	36

Mục lục

Chương 1	Khảo sát và phân tích yêu cầu	1
1.1	Tổng quan vấn đề	1
1.1.1	Bối cảnh chuyển đổi số trong giáo dục	1
1.1.2	Thực trạng và thách thức	2
1.2	Tổng quan về bài toán Sinh câu hỏi	2
1.3	Mục tiêu và phạm vi nghiên cứu	3
1.3.1	Mục tiêu	3
1.3.2	Phạm vi nghiên cứu	4
Chương 2	Công nghệ sử dụng	5
2.1	Tổng quan về kiến trúc công nghệ	5
2.2	Công nghệ phía máy chủ (Backend)	5
2.3	Công nghệ phía người dùng (Frontend)	7
2.4	Các mẫu thiết kế phần mềm được áp dụng	8
2.5	Kiến trúc Lai ghép (Hybrid) Neuro - Symbolic cho Hệ thống Sinh Câu hỏi Tự động	8
2.5.1	Tổng quan về Neuro - Symbolic AI	9
2.5.2	Neural Component - Khả năng Xử lý Ngôn ngữ . . .	9
2.5.3	Symbolic Component - Khả năng Tính toán và Logic	10
2.5.4	Cơ chế tương tác trong hệ thống	11
2.5.5	Lý do lựa chọn kiến trúc cho Đề án	11

Chương 3	Thiết kế và Triển khai hệ thống	13
3.1	Kiến trúc tổng thể	13
3.1.1	Sơ đồ kiến trúc	13
3.1.2	Các thành phần chi tiết	15
3.2	Thiết kế Cơ sở dữ liệu	16
3.2.1	Lược đồ thực thể	16
3.2.2	Tổ chức dữ liệu phi cấu trúc	18
3.3	Thiết kế và Cài đặt Thuật toán Sinh câu hỏi	19
3.3.1	Quy trình sinh dữ liệu	19
3.3.2	Cơ chế Solver và Sandbox	21
3.4	Thiết kế API	22
3.5	Chi tiết giao diện	24
3.5.1	Giao diện Dashboard	24
3.5.2	Giao diện quản lý môn	26
3.5.3	Giao diện quản lý Template	27
3.5.4	Giao diện chỉnh sửa và tạo mới Template	28
3.5.5	Giao diện Danh sách câu hỏi	31
3.5.6	Giao diện Sinh câu hỏi	33
3.5.7	Giao diện Sinh đề	35
	Tài liệu tham khảo	39

Chương 1

Khảo sát và phân tích yêu cầu

1.1 Tổng quan vấn đề

1.1.1 Bối cảnh chuyển đổi số trong giáo dục

Trong thập kỷ qua, giáo dục toàn cầu nói chung và tại Việt Nam nói riêng đã chứng kiến sự chuyển dịch mạnh mẽ sang nền tảng số (EdTech). Sự phổ biến của các hệ thống quản lý học tập (LMS) và các nền tảng thi trực tuyến đã đặt ra nhu cầu cấp thiết về nguồn tài nguyên học liệu số, đặc biệt là **Ngân hàng câu hỏi trắc nghiệm (Question Bank)**.

Để đảm bảo được tính công bằng và đánh giá chính xác năng lực của người học trong các kỳ thi lớn, ngân hàng câu hỏi phải đáp ứng hai điều kiện cốt lõi là **số lượng lớn** và **chất lượng đồng đều**. Để đáp ứng cho việc sinh ra được nhiều mã đề khác nhau nhưng các biến thể của câu hỏi vẫn có độ khó tương đương và đảm bảo tính chính xác về mặt khoa học, giúp hạn chế tối đa việc gian lận và học vẹt.

1.1.2 Thực trạng và thách thức

Hiện nay, việc xây dựng ngân hàng câu hỏi đang được triển khai dựa trên 3 phương pháp chủ yếu:

- Soạn thảo thủ công: Đảm bảo chất lượng chuyên môn nhưng tốn kém thời gian, khó mở rộng quy mô lớn và dễ mắc lỗi sai sót nhỏ do thao tác lặp lại.
- Sinh tự động theo khuôn mẫu: Tốc độ nhanh nhưng cứng nhắc, thường gặp lỗi "ngữ nghĩa toán học" (ví dụ: ra đề bài tuổi con lớn hơn tuổi cha) do thiếu các ràng buộc logic chặt chẽ.
- AI tạo sinh (LLMs): Tuy linh hoạt về ngôn ngữ nhưng thường gặp lỗi "ảo giác" khi tính toán, dẫn đến lời giải sai lệch hoàn toàn dù văn phong trôi chảy.

Từ thực tế như vậy, việc sinh ra một hệ thống lai giữa tư duy toán học chính xác (Symbolic) và khả năng ngôn ngữ linh hoạt của các mô hình AI trở nên khá cần thiết, đáp ứng được nhiều nhu cầu của nhiều đối tượng người sử dụng.

1.2 Tổng quan về bài toán Sinh câu hỏi

Sinh câu hỏi tự động (AQG) được định nghĩa là nhiệm vụ tạo ra một câu hỏi hợp lệ Q từ một nguồn thông tin đầu vào (văn bản C , câu trả lời A), sao cho câu hỏi Q có thể được trả lời chính xác bằng A dựa trên ngữ cảnh C . Về mặt toán học, bài toán này có thể được mô hình hóa như việc tìm chuỗi Q sao cho xác suất có điều kiện $P(Q|C, A)$ là lớn nhất:

$$\hat{Q} = \operatorname{argmax}_Q P(Q|C, A)$$

Lịch sử phát triển của AQG đã trải qua hai giai đoạn chính:

- **Phương pháp dựa trên luật (Rule-based):** Xuất hiện từ những năm 1970, phương pháp này dựa vào các quy tắc biến đổi cú pháp thủ công. Ví dụ, chuyển đổi câu trần thuật thành câu nghi vấn bằng cách thay thế các thực thể bằng từ để hỏi và đảo trật tự từ. Tuy nhiên, phương pháp này phụ thuộc quá nhiều vào các mẫu (templates) định sẵn, thiếu linh hoạt và khó tạo ra các câu hỏi sâu sắc.
- **Phương pháp dựa trên mạng nơ-ron (Neural-based):** Với sự bùng nổ của Deep Learning, bài toán AQG được chuyển sang mô hình Sequence-to-Sequence (Seq2Seq). Mô hình học cách ánh xạ chuỗi văn bản đầu vào sang chuỗi câu hỏi đầu ra thông qua quá trình huấn luyện trên lượng dữ liệu lớn. Đây là hướng tiếp cận hiện đại (State-of-the-Art) cho phép sinh ra các câu hỏi tự nhiên, đa dạng về cấu trúc ngữ pháp và ngữ nghĩa.

1.3 Mục tiêu và phạm vi nghiên cứu

1.3.1 Mục tiêu

Mục tiêu của đề án là xây dựng và triển khai một hệ thống phần mềm có khả năng sinh tự động các biến thể câu hỏi (Question Variants) từ một câu hỏi mẫu, đảm bảo các tiêu chí:

- **Tính chính xác (Accuracy):** Các biến thể được sinh ra phải giải

được và có đáp án đúng, tuân thủ các ràng buộc toán học (nguyên, dương, v.v.).

- **Tính đa dạng (Diversity):** Các biến thể không chỉ thay đổi về số liệu mà còn thay đổi về ngữ cảnh, cách diễn đạt đề bài nhờ vào sức mạnh của Gemini AI.

1.3.2 Phạm vi nghiên cứu

Trong khuôn khổ Đề án 2, hệ thống sẽ tập trung vào:

- **Miền kiến thức:** Các bài toán Đại số và Giải tích cơ bản (Phương trình, Hệ phương trình, Đạo hàm, Tích phân...) ở bậc Trung học phổ thông và Đại học đại cương.
- **Loại câu hỏi:** Trắc nghiệm khách quan.
- **Công nghệ:** Sử dụng Google Gemini API cho phần xử lý ngôn ngữ và thư viện SymPy (Python) cho phần xử lý toán học.

Chương 2

Công nghệ sử dụng

2.1 Tổng quan về kiến trúc công nghệ

Hệ thống quản lý ngân hàng câu hỏi trắc nghiệm được xây dựng dựa trên mô hình Client-Server với kiến trúc 3 tầng, đảm bảo sự tách biệt rõ ràng giữa giao diện người dùng, logic xử lý nghiệp vụ và lưu trữ dữ liệu. Việc lựa chọn ngăn xếp công nghệ (tech stack) cho dự án được cân nhắc kỹ lưỡng dựa trên các tiêu chí về hiệu năng, khả năng mở rộng và đặc thù của bài toán sinh nội dung toán học tự động. Trong đó, Backend đóng vai trò cung cấp RESTful API và xử lý logic phức tạp, còn Frontend tập trung vào trải nghiệm người dùng và hiển thị chính xác các công thức toán học.

2.2 Công nghệ phía máy chủ (Backend)

Nền tảng phía máy chủ được phát triển bằng ngôn ngữ lập trình Python. Đây là lựa chọn tối ưu cho dự án nhờ vào hệ sinh thái thư viện phong phú, đặc biệt là khả năng hỗ trợ mạnh mẽ cho các tác vụ tính toán khoa học và trí tuệ nhân tạo. Để xây dựng dịch vụ web, dự án

sử dụng **Flask**, một micro-framework nổi tiếng với triết lý "convention over configuration". Khác với các framework đồ sộ như Django, Flask cho phép nhóm phát triển có toàn quyền kiểm soát cấu trúc ứng dụng, giữ cho hệ thống nhẹ và linh hoạt, đồng thời hỗ trợ tốt việc xây dựng các API Endpoints để giao tiếp với Client. Bên cạnh đó, thư viện **Flask-CORS** được tích hợp để giải quyết vấn đề chia sẻ tài nguyên chéo nguồn gốc (Cross-Origin Resource Sharing), đảm bảo sự giao tiếp thông suốt giữa Frontend và Backend trong môi trường phân tán.

Để giải quyết bài toán cốt lõi về xử lý toán học, hệ thống tích hợp thư viện **SymPy**. Đây là thư viện tính toán ký hiệu (symbolic computation) hàng đầu, cho phép thực hiện các phép toán đại số, đạo hàm, tích phân với độ chính xác tuyệt đối thay vì sử dụng các phương pháp số học xấp xỉ có thể gây sai số. Điểm đặc biệt của hệ thống là sự kết hợp giữa thuật toán truyền thống và Trí tuệ nhân tạo thông qua **Google Generative AI (Gemini)**. Mô hình ngôn ngữ lớn này được sử dụng để làm giàu nội dung câu hỏi, tự động sinh ra các lời dẫn tự nhiên và tạo các phương án nhiễu (distractors) dựa trên phân tích lỗi sai thường gặp của học sinh, giúp ngân hàng câu hỏi trở nên đa dạng và có chiều sâu sư phạm. Để đảm bảo tính toàn vẹn của dữ liệu trao đổi giữa các thành phần, thư viện **Pydantic** được sử dụng để kiểm tra (validate) dữ liệu đầu vào và đầu ra, đặc biệt là các cấu trúc dữ liệu JSON phức tạp từ AI trả về.

Về mặt lưu trữ, dự án lựa chọn hệ quản trị cơ sở dữ liệu SQLite. Đây là giải pháp cơ sở dữ liệu quan hệ gọn nhẹ, hoạt động theo cơ chế file-based mà không cần cài đặt máy chủ riêng biệt, rất phù hợp cho việc triển khai ứng dụng ở quy mô vừa và nhỏ. Dù đơn giản, SQLite vẫn đảm bảo đầy đủ các chuẩn SQL và tính toàn vẹn dữ liệu (ACID), đồng thời được truy

xuất thông qua mô hình Repository Pattern để tách biệt logic nghiệp vụ khỏi các câu lệnh truy vấn trực tiếp.

2.3 Công nghệ phía người dùng (Frontend)

Giao diện người dùng được xây dựng trên nền tảng **HTML**, tận dụng các thẻ ngữ nghĩa (semantic tags) để tối ưu hóa cấu trúc trang và khả năng tiếp cận. Để đảm bảo tính thẩm mỹ và tốc độ phát triển, dự án sử dụng **Tailwind CSS**, một framework CSS theo hướng utility-first. Công nghệ này cho phép thiết lập giao diện nhanh chóng, nhất quán và hỗ trợ tốt chế độ tối (Dark mode) cũng như khả năng tương thích trên nhiều kích thước màn hình khác nhau mà không cần viết quá nhiều mã CSS tùy chỉnh.

Về mặt tương tác, thay vì phụ thuộc vào các framework nặng nề như React hay Angular, dự án sử dụng **Vanilla JavaScript (ES6+)**. Lựa chọn này giúp ứng dụng giảm thiểu dung lượng tải, tăng tốc độ phản hồi và giúp lập trình viên kiểm soát trực tiếp các thao tác trên DOM. Các tính năng hiện đại của JavaScript như Async/Await và Fetch API được khai thác triệt để để xử lý các tác vụ bất đồng bộ và gọi API từ Backend một cách hiệu quả.

Do đặc thù hiển thị nhiều công thức toán học và mã nguồn, Frontend tích hợp bộ công cụ hiển thị chuyên biệt bao gồm **KaTeX** và **MathJax**. Trong đó, KaTeX được ưu tiên sử dụng cho các công thức nằm trong dòng (inline) nhờ tốc độ render vượt trội, còn MathJax xử lý các công thức phức tạp dạng khối (display) để đảm bảo chất lượng hiển thị chuẩn xuất bản. Ngoài ra, thư viện **CodeMirror** cũng được tích hợp để cung cấp một trình soạn thảo mã nguồn ngay trên trình duyệt với đầy đủ tính

năng tô màu cú pháp (syntax highlighting) cho ngôn ngữ Python, phục vụ trực tiếp cho tính năng chỉnh sửa template của hệ thống.

2.4 Các mẫu thiết kế phần mềm được áp dụng

Bên cạnh các công nghệ nền tảng, dự án đặc biệt chú trọng việc áp dụng các mẫu thiết kế phần mềm chuẩn mực để đảm bảo mã nguồn dễ bảo trì và mở rộng. Nổi bật nhất là **Repository Pattern**, được sử dụng để tạo ra một lớp trừu tượng giữa logic nghiệp vụ và lớp truy cập dữ liệu. Thay vì để các dịch vụ (Services) giao tiếp trực tiếp với cơ sở dữ liệu qua các câu lệnh SQL thô, hệ thống định nghĩa các lớp Repository (như *QuestionRepository*, *TemplateRepository*) chịu trách nhiệm thực hiện mọi thao tác thêm, sửa, xóa và truy vấn. Cách tiếp cận này giúp logic nghiệp vụ trở nên trong sáng hơn, đồng thời giảm thiểu rủi ro khi có sự thay đổi về công nghệ lưu trữ trong tương lai.

Ngoài ra, hệ thống cũng tổ chức mã nguồn theo **Service Layer Pattern**. Các logic nghiệp vụ phức tạp như quy trình sinh câu hỏi, thuật toán trộn đề hay tích hợp AI được gói gọn trong các Service riêng biệt (ví dụ: *QuestionGenerationService*, *AIEnrichmentService*). Việc tách biệt này giúp mã nguồn tuân thủ nguyên lý đơn trách nhiệm (Single Responsibility Principle), cho phép tái sử dụng logic ở nhiều nơi khác nhau trong ứng dụng mà không gây trùng lặp mã nguồn.

2.5 Kiến trúc Lai ghép (Hybrid) Neuro - Symbolic cho Hệ thống Sinh Câu hỏi Tự động

2.5.1 Tổng quan về Neuro - Symbolic AI

Neuro-Symbolic AI là một mô hình kiến trúc lai ghép, kết hợp sức mạnh của hai trường phái AI chủ đạo: **Mạng nơ-ron nhân tạo (Neural Networks)** và **Trí tuệ nhân tạo Ký hiệu (Symbolic AI)**.

Trong bối cảnh bài toán sinh ngân hàng câu hỏi tự động, đặc biệt là các câu hỏi thuộc lĩnh vực Khoa học (Đại số, Giải tích,...), việc sử dụng đơn lẻ một trong hai phương pháp trên đều bộc lộ những hạn chế rõ rệt:

- **Mô hình thuần Neural (Deep Learning/LLMs):** Có khả năng xử lý ngôn ngữ tự nhiên vượt trội và linh hoạt, nhưng thường gặp vấn đề "ảo giác", thiếu tính chính xác trong tính toán số học và suy luận logic đa bước.
- **Mô hình thuần Symbolic (Rule-based):** Đảm bảo độ chính xác tuyệt đối và tính logic chặt chẽ, nhưng cứng nhắc, khó mở rộng (scale) và hạn chế trong việc hiểu ngữ cảnh ngôn ngữ tự nhiên đa dạng.

Kiến trúc Neuro-Symbolic ra đời để tận dụng ưu điểm của cả hai: khả năng **"Học"** và **"Nhận thức"** (Perception) của Neural Networks kết hợp với khả năng **"Suy luận"** (Reasoning) và **"Tính toán"** của Symbolic AI. Đây là sự mô phỏng gần đúng nhất với tư duy con người theo lý thuyết "System 1" (Tư duy nhanh, trực giác) và "System 2" (Tư duy chậm, logic).

2.5.2 Neural Component - Khả năng Xử lý Ngôn ngữ

Trong hệ thống này, thành phần Neural được đại diện bởi các Mô hình Ngôn ngữ Lớn (Large Language Models - LLMs) như Google Gemini. Vai

trò của thành phần này tập trung vào các tác vụ phi cấu trúc:

- **Hiểu ngữ nghĩa (Semantic Understanding):** Phân tích đề bài đầu vào (dưới dạng văn bản hoặc hình ảnh), nhận diện ý định của câu hỏi, và trích xuất các thành phần quan trọng (biến số, điều kiện, đơn vị đo).
- **Mẫu hóa bài toán (Pattern Recognition):** Tổng quát hóa một bài toán cụ thể thành một dạng mẫu (template) trừu tượng, thay thế các giá trị cụ thể bằng các tham số đại diện.
- **Sinh văn bản tự nhiên (Natural Language Generation):** Tạo ra lời dẫn, ngữ cảnh bài toán mới dựa trên cấu trúc đã được xác định, đảm bảo văn phong mượt mà và tự nhiên.

2.5.3 Symbolic Component - Khả năng Tính toán và Logic

Thành phần Ký hiệu đóng vai trò là "bộ não logic" của hệ thống, thường được hiện thực hóa thông qua các hệ thống đại số máy tính (Computer Algebra Systems - CAS) hoặc các ngôn ngữ lập trình cấu trúc (như Python, SymPy). Nhiệm vụ cốt lõi bao gồm:

- **Biểu diễn tri thức (Knowledge Representation):** Lưu trữ cấu trúc câu hỏi dưới dạng các biểu thức toán học chính xác (ví dụ: công thức LaTeX, cây cú pháp).
- **Ràng buộc logic (Constraint Satisfaction):** Đảm bảo các tham số đầu vào khi được sinh ngẫu nhiên vẫn thỏa mãn điều kiện thực tế của bài toán (ví dụ: chiều dài > 0 , mẫu số khác 0, delta phương trình bậc 2 ≥ 0 để có nghiệm thực).

- **Tính toán chính xác (Deterministic Calculation):** Thực hiện giải bài toán dựa trên các công thức đã định nghĩa để sinh ra đáp án đúng (Ground Truth) và các phương án nhiễu (Distractors) một cách chính xác tuyệt đối, loại bỏ hoàn toàn sai số do dự đoán xác suất của LLMs.

2.5.4 Cơ chế tương tác trong hệ thống

Kiến trúc Neuro - Symbolic trong hệ thống sinh ngân hàng câu hỏi hoạt động theo quy trình tương tác hai chiều:

- **Pha 1: Neural $\rightarrow$ Symbolic (Neuro-to-Symbolic):** Hệ thống sử dụng LLM để "đọc" dữ liệu thô (đề thi cũ), sau đó chuyển đổi chúng thành một Biểu diễn trung gian (Intermediate Representation - IR). IR này thường là cấu trúc dữ liệu có định dạng (như JSON hoặc Python Object), chứa các template văn bản và logic toán học.
 - Ví dụ: Từ câu hỏi "Xe đi 50km/h...", LLM trích xuất: *type: "velocity", value: 50, variable: "v"*.
- **Pha 2: Symbolic $\rightarrow$ Neural (Symbolic-to-Neuro):** Bộ xử lý Symbolic thực hiện các thuật toán sinh dữ liệu (Data Augmentation) trên IR: thay đổi số liệu, tính toán lại kết quả. Kết quả cuối cùng được đưa trở lại hoặc kết hợp với văn bản mẫu để tạo thành câu hỏi hoàn chỉnh.

2.5.5 Lý do lựa chọn kiến trúc cho Đề án

Việc áp dụng kiến trúc Neuro - Symbolic cho đề án "Xây dựng hệ thống sinh ngân hàng câu hỏi tự động" mang lại các lợi ích thiết yếu:

- **Độ tin cậy cao (Reliability):** Khắc phục điểm yếu chí mạng của AI tạo sinh (Generative AI) trong giáo dục là tính sai lệch kiến thức. Đáp án được tính toán bằng thuật toán, không phải do dự đoán từ ngữ.
- **Khả năng mở rộng (Scalability):** Chỉ cần một lượng nhỏ dữ liệu đầu vào, hệ thống có thể sinh ra hàng nghìn biến thể câu hỏi khác nhau nhờ khả năng tham số hóa của Symbolic và khả năng viết lại văn bản của Neural.
- **Tính giải thích được (Explainability):** Hệ thống có thể truy vết được quy trình giải (step-by-step solution) thông qua các bước logic trong module Symbolic, hỗ trợ tốt cho việc tạo lời giải chi tiết cho sinh viên.

Chương 3

Thiết kế và Triển khai hệ thống

3.1 Kiến trúc tổng thể

Hệ thống "MathBank" được xây dựng dựa trên mô hình Client-Server kinh điển, tuân thủ kiến trúc phân tầng nhằm đảm bảo sự tách biệt rõ ràng giữa các thành phần chức năng. Việc chia nhỏ hệ thống thành các tầng độc lập không chỉ giúp mã nguồn trở nên trong sáng, dễ bảo trì mà còn tạo điều kiện thuận lợi cho việc mở rộng hoặc thay thế công nghệ của từng tầng trong tương lai mà không làm ảnh hưởng đến toàn bộ hệ thống. Giao thức giao tiếp chủ đạo giữa các tầng là HTTP/REST API, đảm bảo tính chuẩn hóa trong việc trao đổi dữ liệu.

3.1.1 Sơ đồ kiến trúc

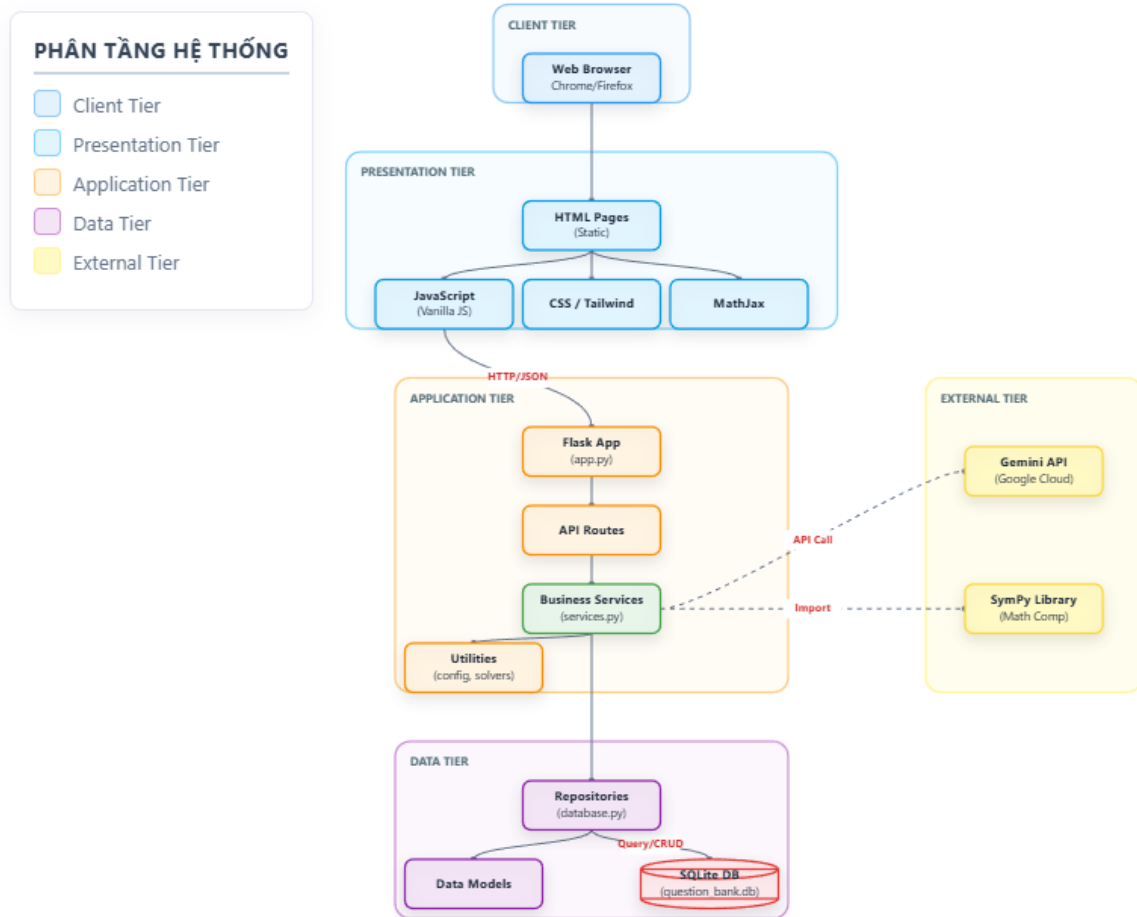
Dựa trên nguyên lý phân tầng, kiến trúc tổng thể của hệ thống bao gồm ba tầng chính, tương tác chặt chẽ với nhau:

Tầng Trình bày (Presentation Layer - Frontend): Đây là điểm tiếp xúc trực tiếp với người dùng cuối (Giáo viên, Quản trị viên). Tầng

này chịu trách nhiệm hiển thị giao diện đồ họa, tiếp nhận thao tác người dùng và trình bày các dữ liệu toán học phức tạp. Được xây dựng trên nền tảng công nghệ web tiêu chuẩn HTML5/CSS3 và JavaScript thuần (Vanilla JS), tầng trình bày đảm bảo hiệu năng cao và tốc độ tải trang nhanh chóng. Giao diện được thiết kế hiện đại nhờ framework **Tailwind CSS**, trong khi việc hiển thị các công thức toán học chuyên sâu được đảm nhiệm bởi sự kết hợp giữa **KaTeX** và **MathJax**.

Tầng Ứng dụng (Application Layer - Backend): Đóng vai trò là trung tâm xử lý nghiệp vụ của toàn bộ hệ thống. Tầng ứng dụng tiếp nhận các yêu cầu từ tầng trình bày, thực thi các thuật toán sinh đề, xử lý logic toán học và điều phối luồng dữ liệu. Nền tảng công nghệ được lựa chọn là **Python** kết hợp với **Flask Framework**, mang lại sự linh hoạt tối đa trong việc xây dựng các API endpoints. Tại đây, các thư viện chuyên biệt như **SymPy** được tích hợp để giải quyết các bài toán đại số, tích phân, đạo hàm một cách chính xác, đóng vai trò là thành phần "Symbolic" trong kiến trúc lai ghép. Ngoài ra, tầng này còn kết nối với các dịch vụ bên ngoài (External Tier) như **Google Gemini API** để thực hiện các tác vụ làm giàu nội dung bằng trí tuệ nhân tạo.

Tầng Dữ liệu (Data Layer): Tầng cuối cùng trong kiến trúc chịu trách nhiệm lưu trữ bền vững và truy xuất dữ liệu. Hệ quản trị cơ sở dữ liệu **SQLite** được lựa chọn nhờ đặc tính gọn nhẹ, không cần cấu hình máy chủ phức tạp (zero-configuration) nhưng vẫn đảm bảo tính toàn vẹn dữ liệu theo chuẩn ACID. Mọi thao tác truy cập dữ liệu đều được thực hiện thông qua các đối tượng Repository, giúp che giấu sự phức tạp của các câu lệnh SQL khỏi logic nghiệp vụ phía trên.



Hình 3.1: Kiến trúc tổng thể

3.1.2 Các thành phần chi tiết

Để hiện thực hóa kiến trúc trên, mã nguồn phía máy chủ (Backend) được tổ chức thành các module chuyên biệt:

Đầu tiên là **Bộ điều khiển API (API Controller)**, thường được định nghĩa trong *app.py*. Thành phần này đóng vai trò là cổng vào (Entry Point) của hệ thống, có nhiệm vụ lắng nghe các HTTP Request từ Frontend, thực hiện xác thực cơ bản (validation) dữ liệu đầu vào và định tuyến yêu cầu đến các dịch vụ xử lý tương ứng.

Tiếp theo là **Lớp Dịch vụ (Service Layer)**, nơi chứa toàn bộ logic nghiệp vụ cốt lõi (Business Logic). Các lớp như *TemplateService*, *QuestionGenerationService* hay *AIEnrichmentService* được xây dựng

để xử lý các quy trình phức tạp như sinh câu hỏi hàng loạt, gọi API AI để tạo đề bài tự nhiên, hay kiểm tra logic toán học của đáp án. Việc tách biệt này giúp mã nguồn tuân thủ nguyên lý đơn trách nhiệm (Single Responsibility Principle), dễ dàng kiểm thử và tái sử dụng.

Cuối cùng là **Lớp Kho chứa**, áp dụng mẫu thiết kế Repository Pattern. Các lớp như *TemplateRepository* hay *QuestionRepository* đóng vai trò là lớp trung gian, trừu tượng hóa toàn bộ các thao tác thêm, sửa, xóa, tìm kiếm với cơ sở dữ liệu. Nhờ đó, logic nghiệp vụ ở tầng trên không bị phụ thuộc trực tiếp vào ngôn ngữ truy vấn SQL, giảm thiểu rủi ro khi có sự thay đổi về cấu trúc lưu trữ.

3.2 Thiết kế Cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống được thiết kế dựa trên mô hình quan hệ (Relational Database Model) sử dụng hệ quản trị **SQLite**. Quyết định lựa chọn SQLite xuất phát từ yêu cầu về một giải pháp lưu trữ file-based linh hoạt, dễ dàng triển khai cục bộ (local deployment) và sao lưu, đồng thời đủ mạnh mẽ để quản lý các ràng buộc toàn vẹn dữ liệu cho một hệ thống quy mô vừa và nhỏ. Việc ánh xạ dữ liệu (ORM) trong ứng dụng được thực hiện thông qua các Python Data Classes, giúp việc thao tác với dữ liệu trở nên trực quan và hướng đối tượng hơn.

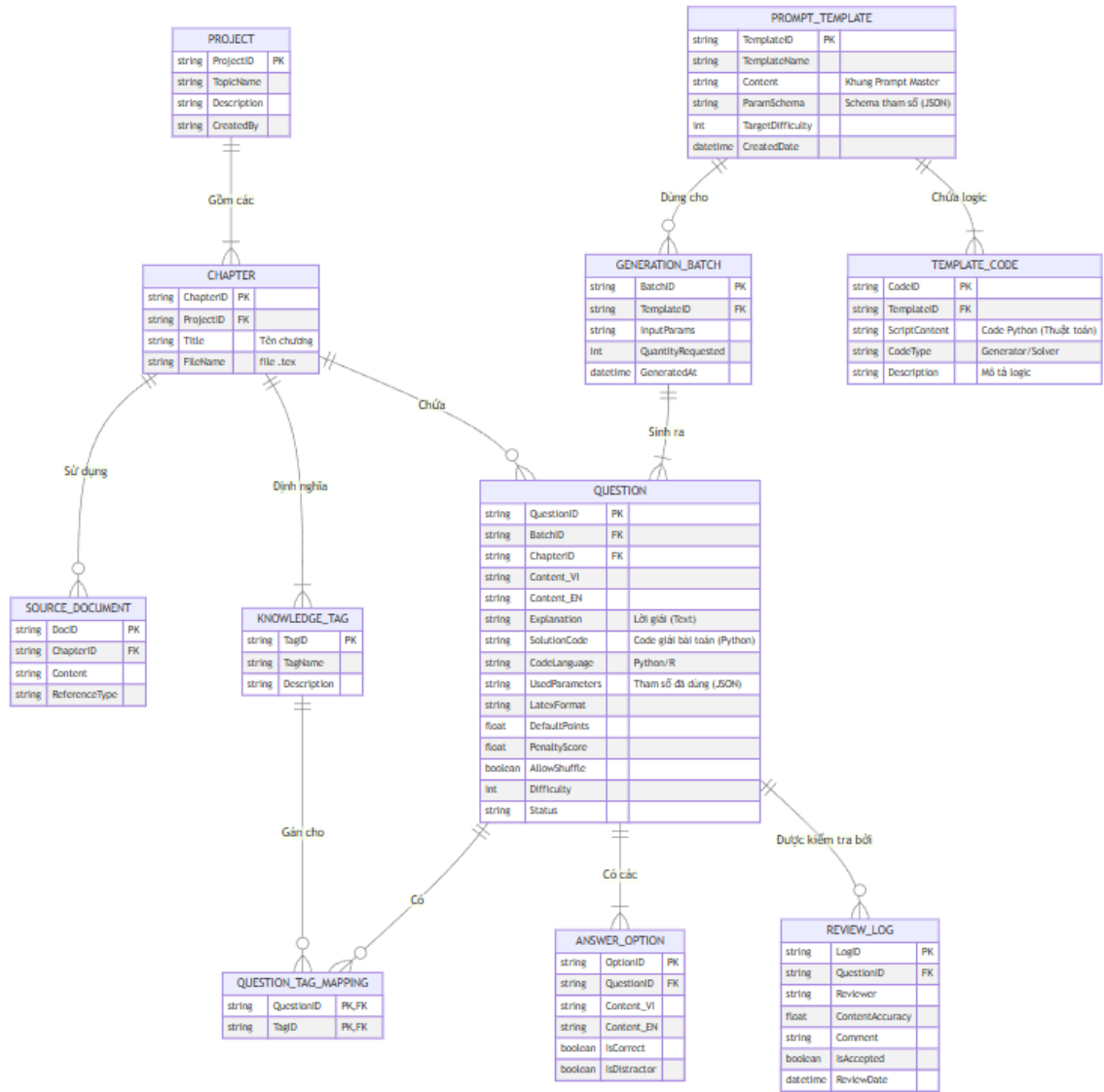
3.2.1 Lược đồ thực thể

Lược đồ cơ sở dữ liệu xoay quanh các thực thể chính, phản ánh quy trình nghiệp vụ từ việc định nghĩa bài toán đến sinh và quản lý câu hỏi:

- **Subject & Category (Môn học & Chủ đề):** Hai bảng này quản lý cấu trúc phân cấp kiến thức của ngân hàng câu hỏi. Thiết

kế sử dụng kỹ thuật tham chiếu đệ quy (self-referencing) thông qua trường *parent_id*, cho phép xây dựng cây thư mục kiến thức có độ sâu tùy ý (Môn học -> Chương -> Mục -> Bài).

- **Template:** Đây là bảng trung tâm lưu trữ siêu dữ liệu (meta-data) của các dạng bài toán. Mỗi bản ghi chứa thông tin định danh, công thức toán học gốc và đặc biệt là logic sinh dữ liệu. Trường *param_schema* trong bảng này đóng vai trò quan trọng trong việc định nghĩa quy tắc biến thiên của các tham số đầu vào.
- **Solver:** Bảng này lưu trữ mã nguồn Python thực thi logic giải toán cho từng Template. Việc tách biệt Solver ra khỏi Template cho phép hệ thống quản lý nhiều phiên bản thuật toán giải khác nhau cho cùng một dạng bài, hỗ trợ tốt cho quá trình nâng cấp và bảo trì thuật toán.
- **Question:** Lưu trữ các thể hiện (instance) cụ thể của câu hỏi sau khi được sinh ra từ Template. Để đảm bảo tính duy nhất và tránh trùng lặp nội dung, hệ thống sử dụng trường *param_hash* – một chuỗi mã hóa đại diện cho bộ tham số đầu vào. Trước khi lưu một câu hỏi mới, hệ thống sẽ kiểm tra mã hash này để đảm bảo không tồn tại một biến thể y hệt trong cơ sở dữ liệu.



Hình 3.2: Sơ đồ kiến trúc dữ liệu

3.2.2 Tổ chức dữ liệu phi cấu trúc

Một thách thức lớn trong bài toán sinh đề trắc nghiệm toán học là sự đa dạng và phức tạp của dữ liệu, khó có thể chuẩn hóa hoàn toàn vào các cột cố định. Để giải quyết vấn đề này, hệ thống áp dụng cách tiếp cận lai (hybrid approach), kết hợp giữa cấu trúc bảng quan hệ chặt chẽ và sự linh hoạt của định dạng JSON cho các trường dữ liệu phi cấu trúc:

Lưu trữ Đáp án (questions.answers): Thay vì tạo bảng riêng cho các phương án lựa chọn, hệ thống lưu trữ toàn bộ 4 phương án A,

B, C, D dưới dạng một chuỗi JSON trong bảng questions. Mỗi đối tượng trong chuỗi này chứa đầy đủ thông tin về nội dung hiển thị (LaTeX), trạng thái đúng/sai (is_correct) và giải thích đi kèm. Cách làm này giúp giảm thiểu số lượng phép kết nối (JOIN) khi truy vấn đề thi, tăng tốc độ phản hồi.

Định nghĩa Tham số (templates.param_schema): Các ràng buộc của biến số đầu vào (như giá trị tối thiểu, tối đa, các giá trị loại trừ, kiểu dữ liệu) cũng được lưu trữ dưới dạng JSON. Điều này cho phép mỗi bài toán có một cấu trúc tham số hoàn toàn khác nhau mà không cần thay đổi cấu trúc bảng cứng (Schema) của cơ sở dữ liệu.

3.3 Thiết kế và Cài đặt Thuật toán Sinh câu hỏi

Phần lõi của hệ thống được xây dựng dựa trên sự phối hợp chặt chẽ giữa các giải thuật định danh (deterministic algorithms) và các mô hình trí tuệ nhân tạo. Để đảm bảo hiệu năng và tính an toàn, hệ thống phân tách rõ ràng giữa quy trình điều phối luồng dữ liệu và cơ chế thực thi mã nguồn tính toán.

3.3.1 Quy trình sinh dữ liệu

Quy trình sinh câu hỏi được thiết kế theo mô hình đường ống xử lý theo lô (Batch Processing Pipeline), cho phép hệ thống tạo ra số lượng lớn các biến thể câu hỏi chỉ trong một lần gọi yêu cầu, tối ưu hóa thời gian phản hồi và chi phí tài nguyên. Quá trình này được điều phối bởi lớp QuestionGenerationService và trải qua ba giai đoạn xử lý tuần tự nghiêm ngặt.

1 . Pha 1: Khởi tạo và Sàng lọc tham số

Dựa trên lược đồ tham số (*param_schema*) được định nghĩa trong Template, thuật toán sẽ sinh ngẫu nhiên các giá trị đầu vào cho bài toán. Thay vì chấp nhận mọi giá trị ngẫu nhiên, hệ thống áp dụng các bộ lọc logic để đảm bảo tính khả thi toán học (ví dụ: mẫu số khác 0, biểu thức dưới dấu căn không âm). Điểm cốt lõi kỹ thuật tại bước này là cơ chế kiểm soát trùng lặp thông qua hàm băm. Mỗi bộ tham số hợp lệ sẽ được băm thành một chuỗi ký tự duy nhất (*param_hash*) và so sánh đối chiếu với cơ sở dữ liệu hiện hành. Nếu phát hiện mã băm đã tồn tại, hệ thống sẽ tự động loại bỏ và kích hoạt lại quy trình sinh tham số, đảm bảo tuyệt đối không có hai câu hỏi trùng lặp trong ngân hàng đề.

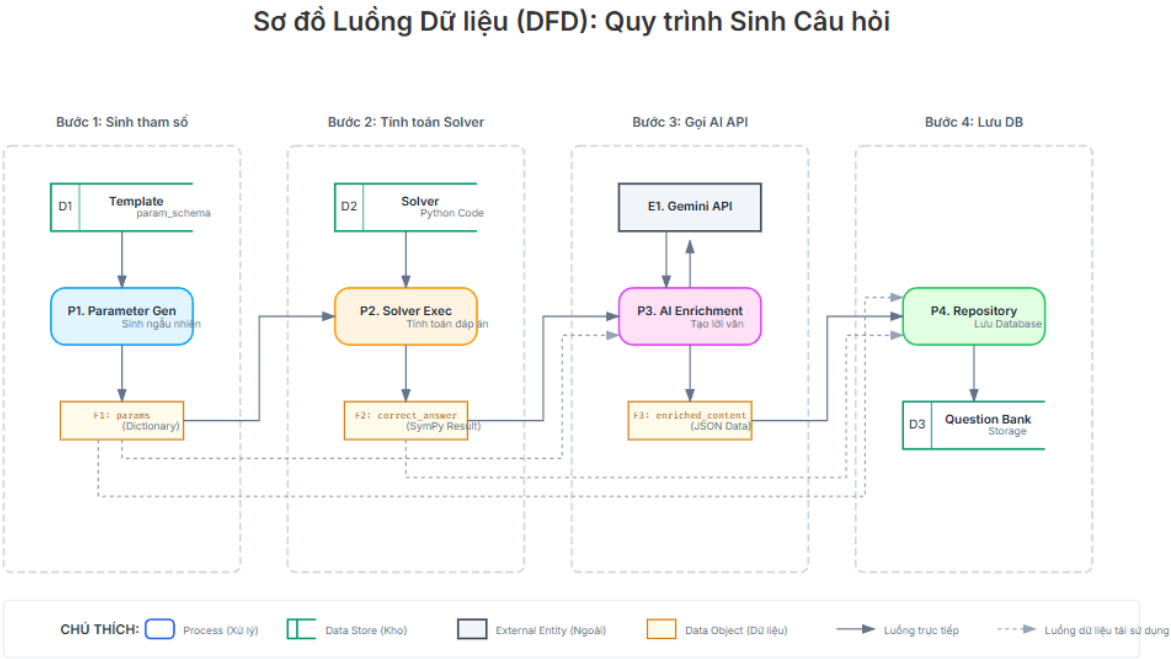
2 . Pha 2: Tính toán hạt nhân (Core Calculation):

Các bộ tham số hợp lệ được chuyển đến thành phần Solver để thực hiện giải toán, thu về kết quả đúng (Ground Truth). Tại đây, thuật toán cũng đồng thời áp dụng các chiến lược sinh phương án nhiễu (Distractor Generation) dựa trên các luật định sẵn được cài đặt trong DistractorService, chẳng hạn như mô phỏng lỗi sai dấu (*sign_error*) hay lỗi tính toán hệ số (*coefficient_error*).

3 . Pha 3: Làm giàu ngữ nghĩa bằng AI (Semantic Enrichment):

Dữ liệu thô gồm tham số và kết quả được đóng gói thành các gói dữ liệu (batches) và gửi đến Google Gemini API. Tại đây, mô hình ngôn ngữ sẽ thực hiện các tác vụ phức tạp như viết lại lời dẫn đề bài cho tự nhiên, bổ sung bối cảnh thực tế và sinh thêm các phương án nhiễu mang tính "lừa" cao dựa trên ngữ nghĩa mà thuật toán thông thường không làm được. Dữ liệu trả về từ AI được kiểm tra nghiêm ngặt về định dạng JSON thông

qua thư viện Pydantic trước khi được lưu trữ chính thức vào cơ sở dữ liệu.



Hình 3.3: Dataflow quy trình sinh câu hỏi

3.3.2 Cơ chế Solver và Sandbox

Trong kiến trúc của hệ thống, việc thực thi mã nguồn sinh câu hỏi (Solver Code) là một tác vụ quan trọng nhưng tiềm ẩn nhiều rủi ro về bảo mật nếu không được kiểm soát chặt chẽ. Do đó, hệ thống đã xây dựng một cơ chế thực thi biệt lập (Sandbox Execution) thông qua lớp SolverService. Thay vì cho phép mã nguồn truy cập trực tiếp vào toàn bộ tài nguyên hệ thống, môi trường thực thi được giới hạn trong một không gian tên cụ thể.

Cơ chế này hoạt động bằng cách khởi tạo một từ điển môi trường (environment dictionary) chỉ chứa các thư viện và hàm được phép sử dụng. Cụ thể, thư viện toán học SymPy được nạp sẵn vào không gian này dưới các định danh phổ biến như `sp`, `sympy`, hay `symbols`. Điều này

cho phép các đoạn mã solver tập trung hoàn toàn vào logic toán học đại số, giải phương trình và tích phân mà không cần khai báo lại các phụ thuộc, đồng thời ngăn chặn tuyệt đối việc gọi các hàm hệ thống nguy hiểm (như truy cập file system hay network) từ phía người dùng. Khi hàm `exec()` được gọi, nó sẽ chạy mã nguồn trong môi trường "hộp cát" này và trả về kết quả thông qua hàm `solve()` được định nghĩa trong template, đảm bảo tính an toàn và ổn định cho máy chủ.

```
195 def execute(self, solver: Solver, params: Dict[str, Any]) -> Any:
196     """Thực thi solver và trả về kết quả"""
197
198     # Tạo namespace với các module cần thiết
199     namespace = {
200
201     }
202
203     # Thực thi code solver
204     try:
205         exec(solver.code, namespace)
206     except Exception as e:
207         raise RuntimeError(f"Lỗi compile solver: {e}")
208
209     # Lấy hàm entry
210     entry_func = namespace.get(solver.entry_function)
211     if not entry_func or not callable(entry_func):
212         raise ValueError(f"Không tìm thấy hàm '{solver.entry_function}' trong solver")
213
214     # Thực thi hàm với params
215     try:
216         result = entry_func(**params)
217         return result
218     except Exception as e:
219         raise RuntimeError(f"Lỗi thực thi solver: {e}")
```

Hình 3.4: Cơ chế Sandbox trong SolverService.execute()

3.4 Thiết kế API

Hệ thống được xây dựng dựa trên nguyên lý kiến trúc RESTful API, đóng vai trò là cầu nối giao tiếp dữ liệu chuẩn hóa giữa Frontend và Backend. Việc thiết kế API tuân thủ nghiêm ngặt các phương thức HTTP chuẩn (GET, POST, PUT, DELETE) giúp đảm bảo tính tường minh và dễ dàng tích hợp mở rộng trong tương lai. Các điểm cuối (endpoints) được tổ chức khoa học theo từng nhóm chức năng nghiệp vụ cụ thể.

Nhóm API quản lý Template (Template Management) cung cấp các phương thức để thao tác với dữ liệu lõi của hệ thống. Các endpoint như */api/get-template* hay */api/save-template* cho phép truy xuất và lưu trữ cấu trúc bài toán, bao gồm cả mã nguồn solver và schema tham số. Đặc biệt, hệ thống hỗ trợ endpoint */api/import-template* để nạp dữ liệu hàng loạt từ định dạng JSON, giúp tăng tốc quy trình khởi tạo ngân hàng câu hỏi ban đầu.

Đối với nghiệp vụ sinh và quản lý câu hỏi, hệ thống cung cấp nhóm API chuyên biệt với khả năng xử lý theo lô (batch processing). Endpoint */api/generate-questions* là một trong những API phức tạp nhất, tiếp nhận cấu hình đầu vào gồm danh sách ID template, số lượng câu hỏi cần sinh, và các ràng buộc về độ khó hoặc miền giá trị tham số. Kết quả trả về không chỉ là danh sách câu hỏi thô mà còn bao gồm các thông tin thống kê chi tiết về số lượng sinh thành công, số lượng trùng lặp bị loại bỏ và trạng thái làm giàu dữ liệu bởi AI. Điều này cung cấp cái nhìn toàn diện cho người quản trị về hiệu suất sinh đề.

Ngoài ra, để hỗ trợ các tiện ích hệ thống, các API như */api/run-code* đóng vai trò quan trọng trong quá trình soạn thảo template, cho phép người dùng chạy thử nghiệm (dry-run) đoạn mã Python solver ngay trên trình duyệt và nhận về kết quả tức thì trước khi lưu vào cơ sở dữ liệu. Mọi dữ liệu trao đổi qua API đều được định dạng chuẩn JSON và được kiểm soát chặt chẽ bởi thư viện Flask-CORS để đảm bảo an toàn bảo mật trong môi trường cross-domain.

```

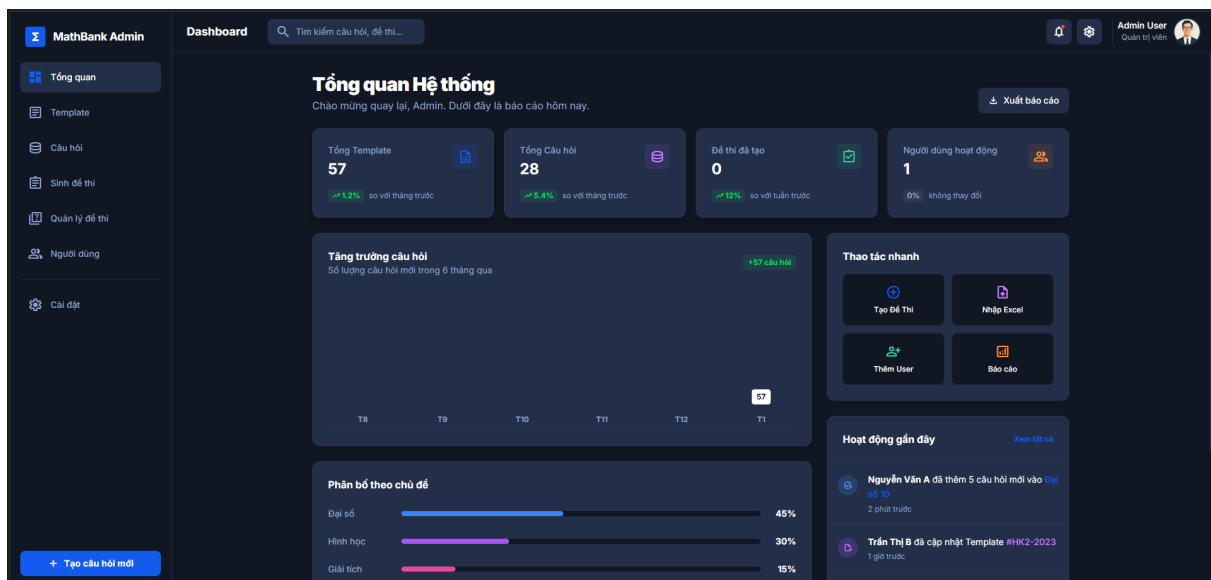
520 # API 2: LƯU TEMPLATE & SINH CÂU HỎI (QUAN TRỌNG NHẤT)
521 # Logic: Lưu Template -> Lưu Code -> Chạy Code sinh câu hỏi -> Lưu vào bảng Question
522 @app.route('/api/save-template', methods=['POST'])
523 > def save_template(): ...
731
732 # API 3: LẤY DANH SÁCH TEMPLATE (Dùng cho trang Template Management)
733 @app.route('/api/get-all-templates', methods=['GET'])
734 > def get_all_templates(): ...
825
826 # API 3.1: LẤY DANH SÁCH TAGS DUY NHẤT (Dùng cho dropdown chuyên đề)
827 @app.route('/api/get-all-tags', methods=['GET'])
828 > def get_all_tags(): ...
845
846 # API 4: LẤY DANH SÁCH CÂU HỎI (Dùng cho trang Assignment)
847 @app.route('/api/get-questions', methods=['GET'])
848 > def get_questions_api(): ...
908
909 # API 5: CHẠY TEST CODE (Nút Run Test)
910 @app.route('/api/run-code', methods=['POST'])
911 > def run_code(): ...
993

```

Hình 3.5: Một số API của Template

3.5 Chi tiết giao diện

3.5.1 Giao diện Dashboard



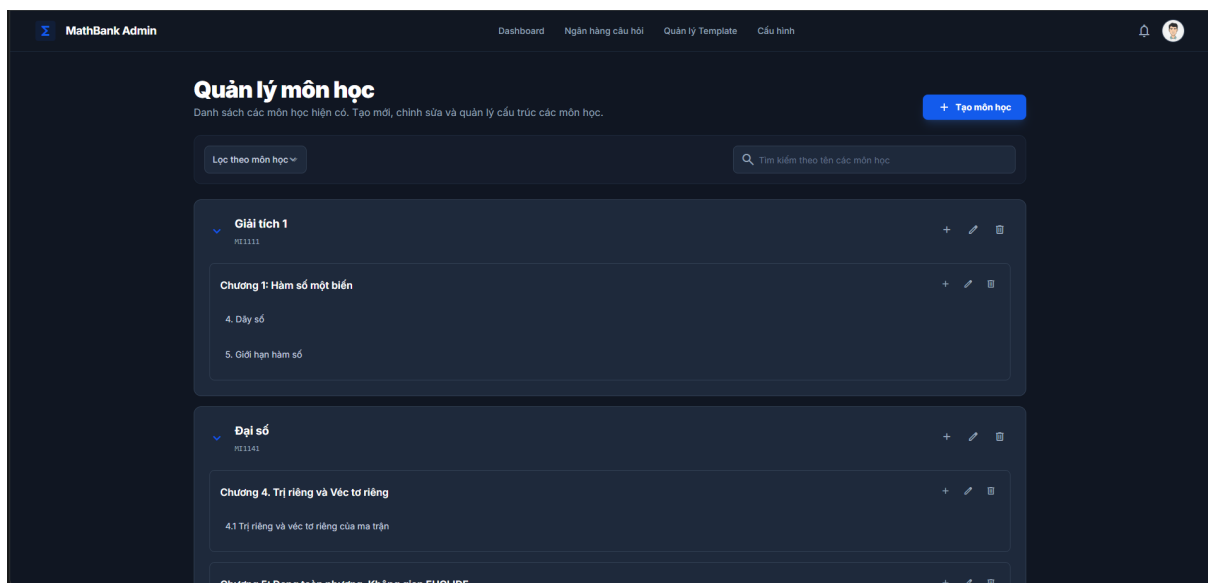
Hình 3.6: Giao diện Dashboard

Giao diện trang Dashboard đóng vai trò là trung tâm điều hành của hệ thống MathBank Admin, được thiết kế theo phong cách hiện đại với gam màu tối (Dark Mode) chủ đạo nhằm tối ưu hóa trải nghiệm thị giác

và giảm mỗi mắt cho người quản trị trong các phiên làm việc kéo dài. Bố cục màn hình được phân chia khoa học thành hai khu vực chính: thanh điều hướng dọc (Sidebar) nằm bên trái và khu vực hiển thị nội dung trung tâm (Main Content Area). Thanh điều hướng cung cấp khả năng truy cập nhanh chóng đến các phân hệ chức năng cốt lõi như quản lý Template, kho câu hỏi, công cụ sinh đề thi tự động và quản lý người dùng, đồng thời tích hợp nút tác vụ nổi bật "Tạo câu hỏi mới" giúp khởi tạo quy trình làm việc tức thì.

Khu vực nội dung chính tập trung vào việc trực quan hóa dữ liệu tổng quan của hệ thống thông qua các thành phần UI được tổ chức chặt chẽ. Hàng trên cùng hiển thị các thẻ chỉ số (Metric Cards) tóm tắt các thông số vận hành quan trọng như tổng số lượng Template (57), quy mô ngân hàng câu hỏi (28), số lượng đề thi đã tạo và số lượng người dùng hoạt động; mỗi chỉ số đều đi kèm với tỷ lệ tăng trưởng so với kỳ trước để hỗ trợ đánh giá hiệu quả. Ngay phía dưới là khu vực biểu đồ, bao gồm biểu đồ đường thể hiện xu hướng tăng trưởng số lượng câu hỏi trong 6 tháng gần nhất và các thanh tiến trình phân tích tỷ lệ phân bố câu hỏi theo từng chủ đề kiến thức (Đại số, Hình học, Giải tích), giúp người quản trị dễ dàng nhận diện sự mất cân đối trong cấu trúc ngân hàng đề. Ngoài ra, giao diện còn tích hợp khối "Thao tác nhanh" để truy cập tất các tính năng thường dùng như nhập liệu từ Excel hay xuất báo cáo, cùng với khu vực "Hoạt động gần đây" đóng vai trò như một nhật ký hệ thống thời gian thực, ghi nhận lịch sử thao tác của các thành viên để đảm bảo tính minh bạch và dễ dàng kiểm soát.

3.5.2 Giao diện quản lý môn



Hình 3.7: Giao diện Quản lý môn học

Tiếp nối từ bảng điều khiển trung tâm, khi người quản trị truy cập vào phân hệ Template, hệ thống sẽ điều hướng đến giao diện "Quản lý môn học". Đây là màn hình đóng vai trò xương sống trong việc tổ chức kiến thức của hệ thống MathBank, nơi định nghĩa cấu trúc phân cấp (hierarchy) cho ngân hàng câu hỏi. Giao diện duy trì ngôn ngữ thiết kế tối giản và hiện đại với tông màu tối đồng bộ, giúp làm nổi bật nội dung chính là cây thư mục kiến thức.

Phần đầu trang được bố trí thanh công cụ điều khiển bao gồm chức năng tìm kiếm thông minh theo tên môn học và bộ lọc nhanh, cho phép người quản trị dễ dàng truy xuất dữ liệu trong bối cảnh số lượng môn học ngày càng mở rộng. Nút chức năng "Tạo môn học" với màu xanh thương hiệu (Primary Blue) được đặt ở vị trí dễ thao tác, đóng vai trò là điểm khởi đầu cho quy trình xây dựng cây kiến thức mới.

Khu vực hiển thị chính được thiết kế dưới dạng danh sách xếp chồng

(Accordion List) có khả năng mở rộng hoặc thu gọn linh hoạt. Mỗi môn học (ví dụ: Giải tích 1, Đại số) được hiển thị như một thẻ độc lập kèm theo mã học phần tương ứng (MI1111, MI1141). Điểm ưu việt của giao diện này là khả năng hiển thị trực quan cấu trúc cha-con đa tầng: từ cấp độ Môn học, đi sâu xuống cấp độ Chương và cuối cùng là các bài học cụ thể hoặc dạng bài như "Dãy số", "Giới hạn hàm số". Tại mỗi cấp độ của cây thư mục, hệ thống tích hợp sẵn các công cụ thao tác nhanh nằm bên phải, bao gồm các biểu tượng trực quan cho phép thêm cấp con (+), chỉnh sửa thông tin (biểu tượng bút chì) hoặc xóa bỏ (biểu tượng thùng rác), giúp việc quản trị cấu trúc đề cương môn học trở nên trực quan và chính xác tuyệt đối.

3.5.3 Giao diện quản lý Template

Quản lý Template
Danh sách các mẫu câu hỏi trắc nghiệm Toán học hiện có. Tạo mới, chỉnh sửa và quản lý cấu trúc đề thi.

MÃ ID	TÊN TEMPLATE	CHUYÊN ĐỀ	ĐỘ KHÓ	CẬP NHẬT	TRẠNG THÁI	HÀNH ĐỘNG
THP-1767775298363-8	4.1 Trị riêng và véc tơ riêng của ma trận - Câu 1	Đại số tuyến tính	Trung bình	08/01/2026	Hoạt động	+
THP-1767775298388-1	4.1 Trị riêng và véc tơ riêng của ma trận - Câu 2	Đại số tuyến tính	Đễ	08/01/2026	Hoạt động	+
THP-1767775298418-2	4.1 Trị riêng và véc tơ riêng của ma trận - Câu 3	Đại số tuyến tính	Đễ	08/01/2026	Hoạt động	+
THP-1767775298438-3	4.1 Trị riêng và véc tơ riêng của ma trận - Câu 4	Đại số tuyến tính	Trung bình	08/01/2026	Hoạt động	+
THP-1767775298458-4	4.1 Trị riêng và véc tơ riêng của ma trận - Câu 5	Đại số tuyến tính	Trung bình	08/01/2026	Hoạt động	+

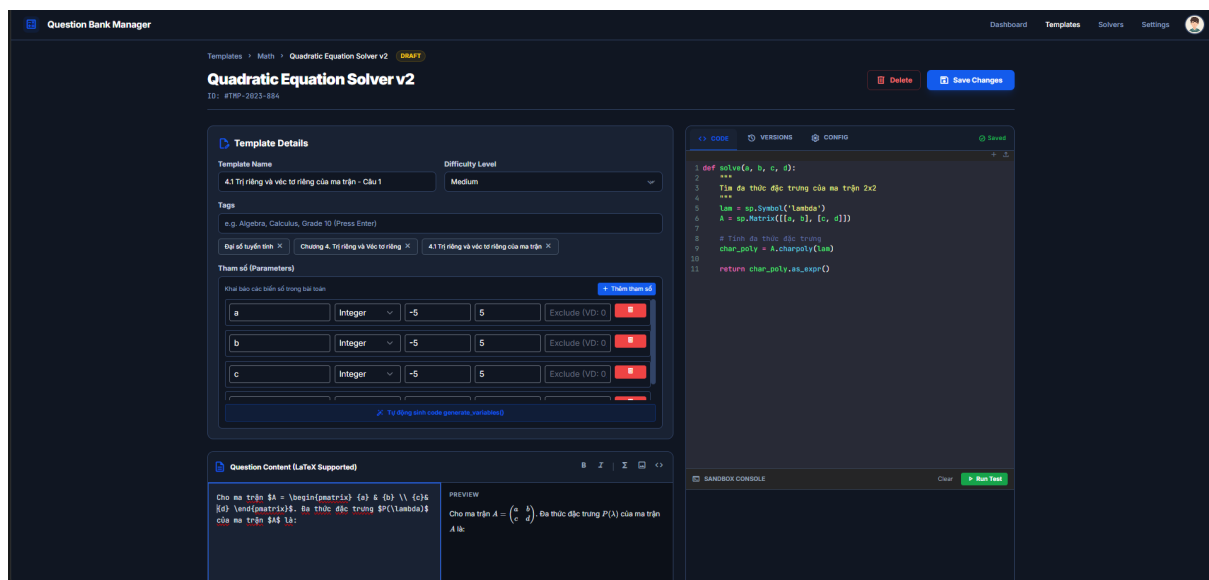
Hình 3.8: Quản lý Template

Tiếp nối luồng thao tác từ cấu trúc môn học, khi người dùng lựa chọn một đơn vị kiến thức cụ thể, hệ thống sẽ chuyển hướng đến giao diện Quản lý Template. Đây là màn hình chức năng đóng vai trò kho lưu trữ

và quản lý tập trung các mẫu câu hỏi (Question Templates) thuộc phạm vi chủ đề đã chọn.

Giao diện được thiết kế để tối ưu hóa khả năng truy xuất dữ liệu với thanh điều hướng phân cấp đặt tại đỉnh trang, giúp người dùng định vị chính xác ngữ cảnh kiến thức hiện tại. Hệ thống trang bị bộ công cụ lọc đa chiều mạnh mẽ (theo khối lớp, độ khó, trạng thái) kết hợp với thanh tìm kiếm từ khóa, cho phép sàng lọc nhanh chóng các mẫu câu hỏi cần thiết. Khu vực trung tâm là bảng dữ liệu hiển thị các thuộc tính quan trọng của Template như mã định danh, tên mẫu, độ khó và trạng thái hoạt động, đồng thời cung cấp các nút tác vụ nổi bật để khởi tạo mới hoặc nhập liệu hàng loạt (Import) từ nguồn bên ngoài.

3.5.4 Giao diện chỉnh sửa và tạo mới Template



Hình 3.9: Giao diện chỉnh sửa và tạo mới Template

Khi người dùng thực hiện tạo mới hoặc chọn chỉnh sửa một mẫu câu hỏi, hệ thống sẽ mở ra giao diện Biên tập Template (Template Editor). Đây là không gian làm việc tích hợp (IDE) trên nền web, được chia thành

hai khu vực làm việc song song giúp tối ưu hóa quy trình từ định nghĩa cấu trúc đến lập trình logic giải toán. Giao diện được phân chia thành các phân hệ chức năng cụ thể như sau:

Khu vực Cấu hình và Nội dung (Cột trái)

Phần bên trái màn hình tập trung vào việc quản lý siêu dữ liệu và giao diện hiển thị của câu hỏi:

- **Thông tin định danh:** Bao gồm các trường nhập liệu cơ bản như Tên Template, Độ khó (*Difficulty Level*) và hệ thống Thẻ (*Tags*). Các thẻ này giúp phân loại kiến thức (ví dụ: Đại số tuyến tính, Ma trận), phục vụ cho việc chỉ mục và tìm kiếm.
- **Cấu hình Tham số (Parameters Schema):** Đây là phân hệ cho phép người dùng định nghĩa các biến số đầu vào cho bài toán. Giao diện cung cấp các dòng nhập liệu trực quan để khai báo tên biến (ví dụ: a, b, c), kiểu dữ liệu (*Integer/Float*) và miền giá trị (*Min/Max*). Tính năng này giúp kiểm soát chặt chẽ biên độ dao động của dữ liệu ngẫu nhiên được sinh ra.
- **Trình soạn thảo nội dung (Content Editor):** Khu vực này hỗ trợ cú pháp LATEX đầy đủ để soạn thảo đề bài. Một tính năng nổi bật là cơ chế **Live Preview** (Xem trước thời gian thực) được đặt ngay bên cạnh khung soạn thảo. Khi người dùng nhập công thức toán học, giao diện sẽ hiển thị kết quả render tương ứng ngay lập tức, giúp kiểm soát lỗi hiển thị một cách trực quan.

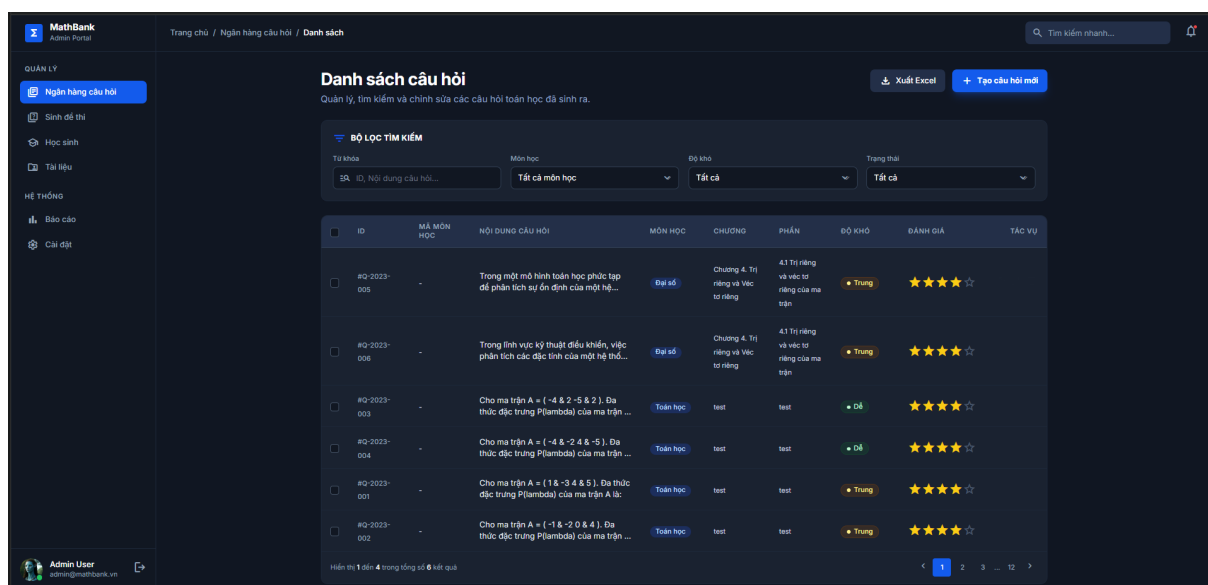
Khu vực Lập trình Logic (Cột phải)

Phần bên phải màn hình đóng vai trò là môi trường phát triển tích hợp (*Code Editor*) dành riêng cho việc viết logic giải toán (*Solver*):

- **Trình biên tập mã nguồn:** Hệ thống tích hợp thư viện **CodeMirror** với khả năng tô màu cú pháp (*Syntax Highlighting*) hỗ trợ ngôn ngữ Python. Tại đây, người dùng xây dựng hàm **solve()** và sử dụng các thư viện toán học chuyên dụng như **SymPy** để xử lý các logic đại số chính xác (ví dụ: tính định thức, tìm đa thức đặc trưng của ma trận).
- **Sandbox Console:** Nằm ở phía dưới cùng của cột phải là bảng điều khiển kiểm thử. Nút chức năng **"Run Test"** cho phép người dùng thực thi thử nghiệm đoạn mã Solver ngay lập tức với các tham số mẫu. Tính năng này hỗ trợ quá trình gỡ lỗi (*Debug*) nhanh chóng, đảm bảo logic hoạt động đúng trước khi lưu trữ vào cơ sở dữ liệu.

Ngoài ra, thanh tiêu đề trên cùng cung cấp các công cụ điều hướng (*Breadcrumbs*) và các nút tác vụ quan trọng như **"Save Changes"** (Lưu thay đổi) hoặc **"Delete"** (Xóa), đảm bảo quy trình quản lý vòng đời của Template được thực hiện trọn vẹn và an toàn.

3.5.5 Giao diện Danh sách câu hỏi



Hình 3.10: Giao diện danh sách câu hỏi

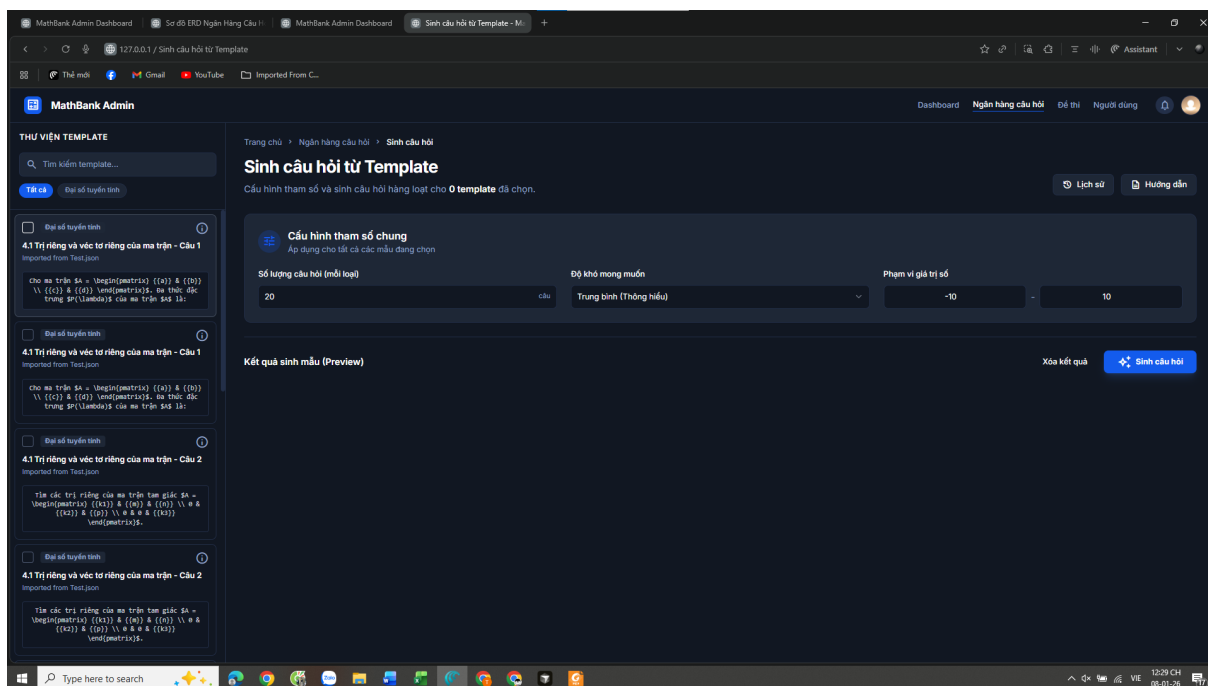
Sau khi các quy trình sinh câu hỏi tự động hoàn tất, toàn bộ dữ liệu đầu ra được tập hợp và quản lý tập trung tại giao diện Danh sách câu hỏi. Đây là màn hình đóng vai trò như một kho lưu trữ số hóa (Digital Repository), cho phép người quản trị thực hiện các nghiệp vụ tra cứu, kiểm duyệt và trích xuất dữ liệu phục vụ cho công tác tạo đề thi. Giao diện vẫn duy trì ngôn ngữ thiết kế tối, nhất quán với hệ thống, giúp làm nổi bật các thông tin quan trọng trong bảng dữ liệu.

Phần đầu trang được bố trí khu vực Bộ lọc tìm kiếm (Search Filter) chuyên sâu, được thiết kế để xử lý lượng dữ liệu lớn. Người dùng có thể sàng lọc câu hỏi theo nhiều tiêu chí kết hợp: từ tìm kiếm cơ bản bằng từ khóa (ID hoặc nội dung văn bản) đến các bộ lọc phân loại theo cấu trúc kiến thức (Môn học), mức độ phức tạp (Độ khó) và tình trạng kiểm duyệt (Trạng thái). Việc bố trí các trường lọc này ngay trên bảng dữ liệu giúp rút ngắn thao tác truy xuất thông tin của quản trị viên. Bên cạnh

đó, các nút chức năng "Xuất Excel" và "Tạo câu hỏi mới" được đặt ở vị trí nổi bật góc phải, hỗ trợ nhanh chóng cho nhu cầu báo cáo thống kê hoặc bổ sung dữ liệu thủ công.

Khu vực trung tâm hiển thị bảng danh sách câu hỏi với các trường thông tin chi tiết được tổ chức khoa học. Mỗi bản ghi câu hỏi được định danh bằng một mã ID duy nhất (ví dụ: #Q-2023-005) kèm theo nội dung đề bài được hiển thị dưới dạng trích dẫn ngắn gọn. Hệ thống cung cấp cái nhìn toàn diện về ngữ cảnh kiến thức của câu hỏi thông qua các cột phân cấp như Môn học, Chương và Phần (Section). Đặc biệt, giao diện tích hợp trực quan hóa các chỉ số định tính: cột "Độ khó" sử dụng nhãn màu (Color-coded tags) để phân biệt mức độ Dễ/Trung bình/Khó, trong khi cột "Đánh giá" sử dụng biểu tượng ngôi sao để thể hiện chất lượng hoặc độ phổ biến của câu hỏi dựa trên phản hồi hệ thống. Cuối cùng, thanh phân trang ở đáy màn hình giúp điều hướng thuận tiện giữa hàng nghìn bản ghi mà không ảnh hưởng đến hiệu năng tải trang.

3.5.6 Giao diện Sinh câu hỏi



Hình 3.11: Giao diện Sinh câu hỏi

Giao diện **Sinh câu hỏi từ Template** đóng vai trò là bảng điều khiển trung tâm để kích hoạt thuật toán sinh dữ liệu tự động, nơi kết nối giữa các mẫu bài toán tĩnh và nhu cầu tạo đề thi động. Màn hình được thiết kế theo bố cục chia đôi (Split-view), tách biệt rõ ràng giữa khu vực chọn nguồn dữ liệu và khu vực cấu hình tham số sinh.

Panel Thư viện Template (Bên trái): Cột bên trái hiển thị danh sách toàn bộ các Template khả dụng trong hệ thống, hoạt động như một thực đơn chọn món. Để hỗ trợ người dùng tìm kiếm nhanh trong kho dữ liệu lớn, hệ thống tích hợp thanh tìm kiếm từ khóa và các tab lọc theo chuyên đề (như "Đại số tuyến tính"). Mỗi thẻ Template trong danh sách không chỉ hiển thị tên gọi mà còn cung cấp một bản xem trước nội dung đề bài dưới dạng LaTeX, giúp người dùng xác nhận chính xác dạng bài cần sinh. Cơ chế hộp kiểm (Checkbox) cho phép chọn đa mục tiêu, nghĩa

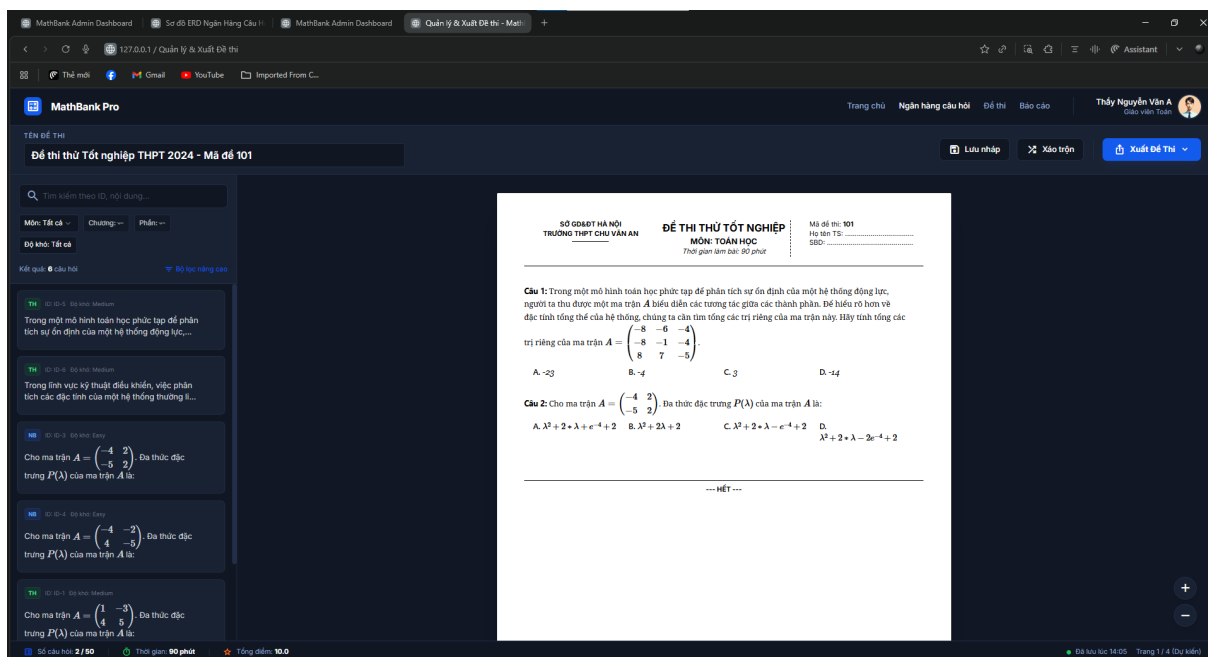
là người quản trị có thể chọn cùng lúc nhiều dạng bài khác nhau để thực hiện sinh hàng loạt trong một lần thao tác.

Khu vực Cấu hình tham số chung (Trung tâm): Phần nội dung chính là nơi người dùng thiết lập các biến số đầu vào cho thuật toán sinh. Panel "Cấu hình tham số chung" cung cấp ba trường dữ liệu quan trọng quyết định đầu ra của sản phẩm:

- **Số lượng câu hỏi:** Cho phép định lượng số biến thể cần tạo ra cho mỗi loại Template đã chọn (mặc định là 20 câu), giúp kiểm soát quy mô ngân hàng đề.
- **Độ khó mong muốn:** Menu thả xuống cho phép gán nhãn độ khó (ví dụ: Trung bình - Thông hiểu), tham số này sẽ được hệ thống ghi nhận để phục vụ cho việc trộn đề thi sau này.
- **Phạm vi giá trị số:** Hai trường nhập liệu Min/Max (ví dụ: từ -10 đến 10) đóng vai trò là ràng buộc (Constraints) cho thuật toán Random, đảm bảo các con số trong đề bài được sinh ra nằm trong ngưỡng tính toán phù hợp với trình độ học sinh.

Phía dưới cùng là khu vực "**Kết quả sinh mẫu (Preview)**", hiện tại đang ở trạng thái chờ. Sau khi người dùng nhấn nút tác vụ chính "**Sinh câu hỏi**" màu xanh nổi bật, hệ thống sẽ gửi yêu cầu về Backend, xử lý và trả về kết quả xem trước tại đây để người dùng kiểm tra xác suất lỗi trước khi chính thức lưu vào cơ sở dữ liệu.

3.5.7 Giao diện Sinh đề



Hình 3.12: Giao diện Sinh đề thi

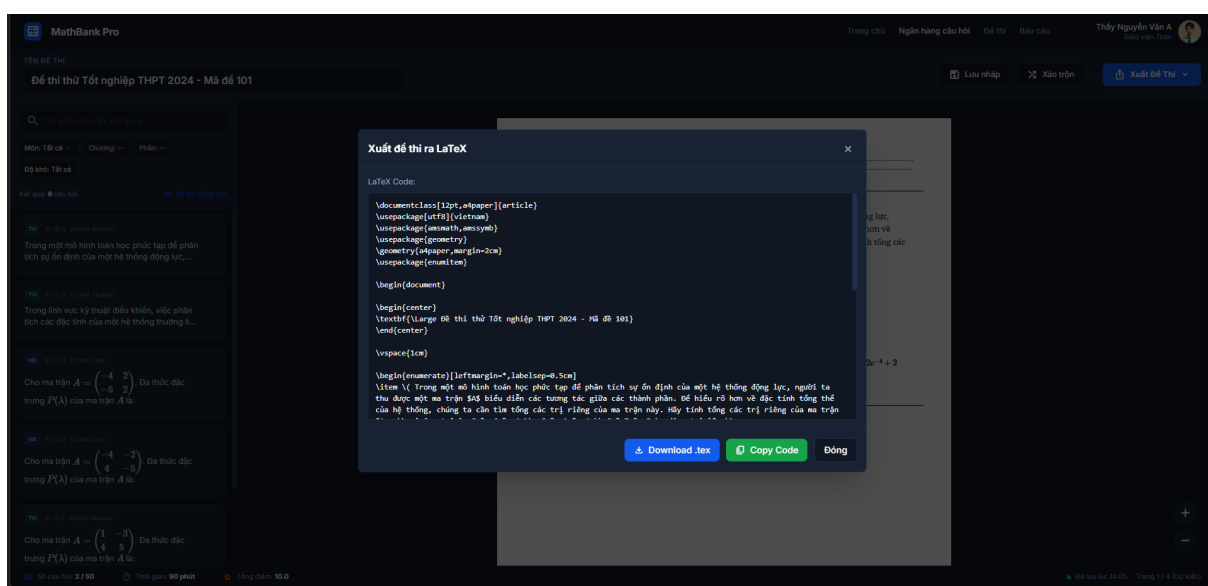
Sau khi đã chuẩn bị đầy đủ ngân hàng câu hỏi, quy trình cuối cùng và quan trọng nhất là tổng hợp thành đề thi hoàn chỉnh. Giao diện **Soạn thảo Đề thi** được thiết kế theo mô hình tương tác trực quan "WYSIWYG" (What You See Is What You Get), giúp người dùng hình dung chính xác kết quả bản in ngay trong quá trình biên tập. Bố cục màn hình được phân chia thành hai luồng công việc rõ rệt: kho dữ liệu nguồn bên trái và bảng thiết kế đề thi bên phải.

Panel Kho câu hỏi (Bên trái): Khu vực này đóng vai trò là "nguyên liệu" để xây dựng đề thi. Tại đây, hệ thống liệt kê danh sách các câu hỏi khả dụng kèm theo bộ lọc đa tầng (Môn, Chương, Phần, Độ khó) giúp giáo viên nhanh chóng tìm kiếm các nội dung phù hợp. Mỗi thẻ câu hỏi hiển thị tóm tắt nội dung đề bài (có render công thức toán) và các nhãn siêu dữ liệu (Metadata) như ID và độ khó. Thanh trạng thái

ở góc dưới cùng bên trái liên tục cập nhật số lượng câu hỏi đã chọn (ví dụ: 2/50), giúp giáo viên kiểm soát độ dài của đề thi theo đúng cấu trúc ma trận đề mong muốn.

Khu vực Thiết kế và Xem trước (Bên phải): Đây là điểm nhấn công nghệ của hệ thống, mô phỏng chính xác giao diện của một đề thi giấy tiêu chuẩn.

- **Định dạng chuẩn hóa:** Hệ thống tự động tạo Header (Tiêu đề) theo quy chuẩn hành chính (Sở GD&ĐT, Trường, Mã đề, SBD) và Footer (Kết thúc trang), giúp giáo viên tiết kiệm thời gian căn chỉnh văn bản.
- **Hiển thị toán học chuyên sâu:** Các nội dung phức tạp như ma trận, hệ phương trình hay biểu thức tích phân được hiển thị sắc nét, căn chỉnh lề chính xác tuyệt đối nhờ sự tích hợp sâu của MathJax. Các phương án trắc nghiệm A, B, C, D được dàn trang tự động, đảm bảo tính thẩm mỹ.



Hình 3.13: Xuất đề thi ra Latex

Thanh công cụ và Tiện ích: Phía trên cùng là thanh công cụ điều khiển với các chức năng nghiệp vụ cốt lõi. Người dùng có thể đặt tên đề thi, lưu bản nháp hoặc sử dụng tính năng "Xáo trộn" để tự động tạo ra các mã đề hoán vị khác nhau từ cùng một tập câu hỏi gốc. Nút "Xuất Đề Thi" ở góc phải cho phép trích xuất kết quả ra các định dạng phổ biến như pdf và Latex. Ngoài ra, thanh trạng thái dưới cùng cung cấp thông tin thời gian làm bài (90 phút) và tổng điểm, hỗ trợ giáo viên cân đối lượng kiến thức phù hợp với thời lượng kiểm tra.

Kết luận

Đồ án "Xây dựng hệ thống sinh ngân hàng câu hỏi tự động" đã hoàn thành mục tiêu nghiên cứu và triển khai một giải pháp công nghệ toàn diện, giải quyết được bài toán khan hiếm dữ liệu luyện thi chất lượng cao trong bối cảnh chuyển đổi số giáo dục.

Về mặt công nghệ, hệ thống đã chứng minh tính hiệu quả của kiến trúc lai ghép (Neuro-Symbolic AI). Sự kết hợp giữa khả năng tính toán chính xác tuyệt đối của SymPy (Symbolic) và khả năng xử lý ngôn ngữ tự nhiên linh hoạt của Google Gemini (Neural) đã khắc phục được điểm yếu cố hữu của từng phương pháp khi đứng riêng lẻ: loại bỏ được ảo giác tính toán của AI tạo sinh và sự cứng nhắc về văn phong của các thuật toán sinh mẫu truyền thống.

Về mặt kiến trúc phần mềm, việc áp dụng mô hình phân lớp (Layered Architecture) cùng các mẫu thiết kế chuẩn mực như Repository Pattern và Service Layer đã tạo ra một mã nguồn trong sáng, dễ bảo trì và mở rộng. Hệ thống Backend với Flask và SQLite hoạt động ổn định, đáp ứng tốt các yêu cầu về hiệu năng xử lý lô (batch processing) và quản lý dữ liệu. Giao diện Frontend hiện đại, thân thiện với người dùng cũng là một điểm sáng, giúp giảm thiểu rào cản kỹ thuật cho giáo viên trong quá trình biên soạn ngân hàng câu hỏi.

Tài liệu tham khảo

Tiếng Việt

- [1] Nguyễn Kim Anh (2014). *Giáo trình Nguyên lý Hệ quản trị Cơ sở dữ liệu*. Nhà xuất bản Đại học Bách Khoa Hà Nội.

Tiếng Anh

- [2] Kurdi, G., Leo, J., Parsia, B., Sattler, U., & Al-Emari, S. (2020). *A Systematic Review of Automatic Question Generation for Educational Purposes*. International Journal of Artificial Intelligence in Education, 30, 121–204.
- [3] Garcez, A. d., & Lamb, L. C. (2023). *Neurosymbolic AI: The 3rd Wave*. Artificial Intelligence Review.
- [4] Sarker, M. K., Zhou, L., Eberhart, A., & Hitzler, P. (2021). *Neuro-symbolic artificial intelligence: Current trends*. arXiv preprint arXiv:2105.05330.
- [5] Pan, L., Lei, W., Chua, T. S., & Kan, M. Y. (2019). *Recent Advances in Neural Question Generation*. arXiv preprint arXiv:1905.08949.
- [6] Meurer, A., et al. (2017). *SymPy: symbolic computing in Python*. PeerJ Computer Science, 3, e103.

- [7] Gemini Team, Google. (2023). *Gemini: A Family of Highly Capable Multimodal Models*. arXiv preprint arXiv:2312.11805.
- [8] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., ... & Zhou, D. (2022). *Chain-of-thought prompting elicits reasoning in large language models*. Advances in Neural Information Processing Systems (NeurIPS).
- [9] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
- [10] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.
- [11] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.